Scoped Enumeration Stringification and Annotation-Driven Defaultable Formatting with C++26 Static Reflection

Extensions for `std::to_string` and `std::formatter`

Document #:	D0000R0 [Latest] [Status]
Date:	2026-11-08
Project:	Programming Language C++
Audience:	Library Evolution Working Group (LEWG)

Reply-to:	Jeremy Murphy <jeremy dot r dot murphy at protonmail dot com>

Contents

1 Abstract **2**

2 Revision History **2**
2.1 R0 . . . 2

3 Motivation and Scope **2**
3.1 Motivation . . . 2
3.2 Scope . . . 2
3.3 Examples . . . 2

4 Impact on the Standard **2**

5 Design Decisions **3**
5.1 Overview . . . 3
5.2 Alternatives Considered . . . 3
5.3 Naming and Interface Design . . . 3
5.4 Constraints and Limitations . . . 3

6 Proposal **3**
6.1 Overview . . . 3
6.2 Stringification . . . 3
6.3 Formatting Semantics . . . 3
6.4 Enumerator Aliases . . . 3
6.5 Annotations . . . 3
6.5.1 `std::annotations::formattable_with_default` . . . 3
6.5.2 `std::annotations::formattable` . . . 3
6.5.3 `std::annotations::canonical` . . . 3
6.6 Examples . . . 3

7 Implementation Experience **4**
7.1 Reference Implementation . . . 4
7.2 Implementation Status . . . 4
7.3 Testing . . . 4

8 Feature-Test Macro **4**

9 Proposed Wording **4**
9.1 Header Changes . . . 4
9.2 Library Specification Changes . . . 4
9.3 Feature-Test Macro Changes . . . 4

10 Acknowledgements 4

11 References 4

1 Abstract

This paper proposes …

Introduce the problem domain and the proposed facility.

Explain the relationship between this proposal and existing C++ formatting facilities.

2 Revision History

2.1 R0

— Initial revision.

3 Motivation and Scope

3.1 Motivation

Describe the problem being addressed and why existing facilities are insufficient.

3.2 Scope

Define what this proposal intends to address and what remains outside its scope.

3.3 Examples

Provide motivating examples demonstrating the need for the proposed functionality.

```
// Example usage goes here.
```

Before (Current C++)	After (With Proposal)

`// Motivating example showing current verbose, error-prone, or inefficient code`
`#include <iostream>`
 	
`void legacy_way() {`
`  // Current approach`
`}`
	`// Clean code using your proposed feature`
	`void proposed_way() {`
	`  // Proposed syntax or library feature`
	`}`

`void legacy_way() {`
`  // Current approach`
`}`
	`void proposed_way() {`
	`  // Proposed syntax or library feature`
	`}`

4 Impact on the Standard

Describe the impact on the C++ standard, including:

— Which existing library components are affected.
— Whether new library components are introduced.
— Whether the proposal changes existing behavior.
— Whether the proposal requires language changes.
— Compatibility and implementation considerations.

5 Design Decisions

5.1 Overview

Explain the overall design and its guiding principles.

5.2 Alternatives Considered

Describe alternative approaches and the rationale for accepting or rejecting them.

Option A	Option B
`//chose this one`	⃠ **Not proposed** `//didn't do this`

5.3 Naming and Interface Design

Explain the public interface and naming decisions.

5.4 Constraints and Limitations

Describe any deliberate restrictions or limitations.

6 Proposal

6.1 Overview

Summarize the proposed functionality.

6.2 Stringification

6.3 Formatting Semantics

Describe the formatting behavior, including:

— Default representations.
— Format-specification syntax.
— Qualified and unqualified representations.
— Integer representations.
— Forwarded format specifications.
— Type-name representations.

6.4 Enumerator Aliases

Describe canonical enumerator selection and alias handling.

6.5 Annotations

Describe the annotation facilities and their semantics.

6.5.1 `std::annotations::formattable_with_default`

6.5.2 `std::annotations::formattable`

6.5.3 `std::annotations::canonical`

6.6 Examples

Provide representative examples demonstrating the proposed functionality.

7 Implementation Experience

Describe existing implementations, compiler support, implementation experience, and testing.

7.1 Reference Implementation

Describe the reference implementation.

7.2 Implementation Status

Describe compiler and library support.

7.3 Testing

Describe the testing strategy and relevant test coverage.

8 Feature-Test Macro

Specify the proposed feature-test macro and its value.

9 Proposed Wording

This section contains the proposed changes to the C++ working draft.

Use markup tags like `[added text]{.add}`, `[removed text]{.rm}`, and `[removed text](added text){.sub}`.

> Modify Section [general.description] as follows:
>
> The behavior of the function shall be modified to support the new functionality ~~throw an exception~~.

9.1 Header Changes

[Insert proposed wording.]

9.2 Library Specification Changes

[Insert proposed wording.]

9.3 Feature-Test Macro Changes

[Insert proposed wording.]

10 Acknowledgements

Acknowledge contributors and reviewers.

11 References

— [Nxxxx] ISO/IEC 14882: Programming Languages — C++.
— [Pxxxx] Related WG21 proposal.
— [Pxxxx] Related WG21 proposal.